BCSE302P – Database Systems Lab

D2 Slot – L11+L12

BlueLedger

Community Mangrove Reforestation & Carbon-Credit Ledger

Project Proposal & Database Design Document

Assessment 6 — 10 Marks

Course	BCSE302P – Database Systems Lab
Submission	1 of 3 — Project Proposal & Database Design Document
Submission Date	29 September 2026
Repository	Blue-Ledger (GitHub) — Android / Kotlin / Jetpack Compose
Student Name	[To be filled by student]
Registration Number	[To be filled by student]
Faculty In-Charge	[To be filled by student]

This document has been generated by analysing the actual BlueLedger Android codebase (Kotlin, Jetpack Compose, DataStore) and proposes the database design required to satisfy BCSE302P Assessment 6. Fields shown in brackets are left for the student to complete.

Table of Contents

1. Project Title 3

2. Introduction and Background 3

3. Problem Statement 3

4. Objectives 3

5. Scope of the Project 4

6. Requirement Analysis 4

6.1 Functional Requirements 4

6.2 Non-Functional Requirements 5

7. Database Selection and Justification 5

7.1 Comparison with alternatives 6

7.2 Advantages and limitations 6

7.3 Integration with the existing stack 6

8. Data Model / Database Design 8

8.A Entity-Relationship Diagram 8

8.B Entity Descriptions 8

8.C Relational Schema (SQL DDL) 10

8.D Keys and Constraints 12

8.E Normalization 12

9. System Architecture / Workflow Diagram 14

A. Existing vs Proposed System 15

B. Proposed Implementation Plan 15

C. Assessment Readiness Checklist 15

D. Future Enhancements 16

1. Project Title

BlueLedger: A Relational Database System for Verified Mangrove Reforestation Tracking and Carbon-Credit Accounting

BlueLedger is an existing offline-first Android application (Kotlin, Jetpack Compose) that lets field users record mangrove-planting plots and automatically computes carbon credits for them. This document proposes adapting its ad-hoc local storage into a proper relational database so the same real-world workflow gains referential integrity, auditable transactions, and queryable history.

2. Introduction and Background

Mangrove forests are among the most effective natural carbon sinks, and community-driven reforestation programs increasingly rely on digital tools to record planting activity and convert verified effort into tradable carbon credits. **BlueLedger** (this repository) is an Android application built for exactly this purpose: field users log in, record a planting plot (GPS location, species, area in hectares, photo evidence), and the app computes a carbon-credit amount using a species-specific per-hectare rate with area-based bonus multipliers.

The application already implements a working mobile UI (splash, onboarding, login/signup, a four-tab home screen – Dashboard, Upload, Wallet, Profile), device GPS capture, an image picker for plot evidence, and a deterministic carbon-credit formula. However, as documented in the project's own development notes, it has **no real database**: all data is serialized into unstructured text and held in the device's local application storage. This document treats that gap as the starting point for the required database design work.

3. Problem Statement

Community reforestation programs need a trustworthy, auditable record of who planted what, where, and how many carbon credits that effort is worth. In the current BlueLedger implementation this record is fragile by construction:

- **No relational structure.** The application currently keeps the entire list of planting projects as one block of unstructured text, and the current user's details as another single block. There is no per-record identity the storage understands, no foreign keys, and no way to query a subset of records without loading everything into memory first.
- **Unsafe concurrent writes.** Adding a new plot means reading the whole project list, appending one entry in memory, and writing the whole list back. Two near-simultaneous submissions can race and silently overwrite each other — there is no transactional guarantee.
- **Business rules hidden in code, not data.** The species-to-carbon-rate mapping (Rhizophora = 14, Avicennia = 12, Bruguiera = 10 credits/hectare) is hardcoded into the app's logic rather than stored as data. Adding a species or correcting a rate requires a new app release, and there is no historical record of which rate was in effect when a given credit was minted.
- **No audit trail for credits.** The minted-credit amount is baked directly into each project record at upload time. There is no separate ledger of mint/redeem events, so a future credit-redemption feature cannot be implemented safely.
- **Single-user, unauthenticated storage.** Login/signup currently accepts any non-empty credentials and stores exactly one user record per device — there is no user table, no password verification, and no way for a plot to be attributed to a specific verified contributor.

A structured database directly addresses each of these problems: it gives every plot, user, species rate, and credit event a durable, constrained, queryable identity, and lets the minting rule live as data rather than as compiled logic.

4. Objectives

- Replace the current unstructured, single-block storage of user and project data with a normalized relational schema (five tables) accessed through a proper embedded database, without changing the app's existing screens, navigation, or carbon-credit formula.
- Enforce referential integrity with primary/foreign-key constraints so no *Project* can reference a non-existent *User* or *Species*, and no *CreditTransaction* can reference a non-existent *Project*.
- Externalize the species-to-credit-rate mapping (currently hardcoded into the app's logic) into a queryable *Species* reference table.
- Introduce an append-only *CreditTransaction* ledger so minted and (future) redeemed credits form an auditable history, independent of the current wallet balance shown on screen.
- Support indexed, filterable queries (by user, species, status, date range) so the Dashboard and Wallet screens no longer need to load the entire upload history into memory on every screen open.
- Document the schema to at least Third Normal Form (3NF) with a concrete example drawn from this project's own data (the species-rate transitive dependency).

5. Scope of the Project

Category	Items
Already implemented (existing application, unchanged)	Mobile UI: splash, onboarding, login/signup, Dashboard/Upload/Wallet/Profile screens · in-app navigation · GPS capture for plot location · image picker for plot photos · the carbon-credit calculation formula (species rate × hectares × area bonus) · saved “remember me” and language preference
Included in this proposal (database-design scope)	Relational schema for User, Community, Species, Project and CreditTransaction · a proper database layer replacing the current unstructured local storage · CRUD operations backed by SQL · aggregate queries for Dashboard/Wallet totals · project verification status field
Existing features that the new schema must not break	The species-selection dropdown on the Upload screen, the bonus-multiplier rule (≥10 ha → 1.2×, ≥5 ha → 1.1×), and the wallet-summary figures shown on the Wallet screen
Excluded from current scope	Cloud/server backend and network sync · blockchain-based credit minting · real payment or credit-redemption processing · a dedicated admin/verifier screen (the schema reserves a status field for this, but building the review screen is future work) · multi-device data synchronization

6. Requirement Analysis

6.1 Functional Requirements

User roles

- **Registered Contributor** (implemented today as a mock login): creates an account, submits planting projects, views their own dashboard and wallet.
- **Verifier / Admin** (proposed, enabled by a status field on each project but out of scope to build a screen for in this phase): would approve or reject submitted projects before credits are minted.

Data to be stored

- User accounts (email, username, phone, credentials, optional community membership)
- Communities/cooperatives enrolled in the program
- Reference data for mangrove species and their carbon-credit rate per hectare
- Planting projects (plot id, species, hectares, GPS coordinates, photo evidence, verification status)

- An append-only ledger of credit-minting and credit-redemption events

CRUD operations required

Entity	Create	Read	Update	Delete
User	Sign up	Login / profile view	Edit profile	Not exposed (retain audit trail)
Community	Program admin enrolls a community	Dashboard “total communities” stat	Update region/name	Rare / admin-only
Species	Seed reference rows	Populate species dropdown	Adjust credit rate	Not exposed (rates are historical)
Project	Upload screen → new plot	Dashboard / Wallet history	Edit before verification	Delete a draft/pending project
CreditTransaction	System-generated on mint / redeem	Wallet ledger view	Immutable (append-only)	Never deleted (audit requirement)

Data retrieval, filtering, relationships and reporting

- Filter a user's projects by species, verification status, or date range.
- Sort a user's credit-transaction history chronologically (Wallet screen).
- Look up a project's species rate *at the time it was planted* via the Species reference table, rather than recomputing it from constants buried in application logic.
- Insert a Project and its initial MINT CreditTransaction as a single atomic transaction so credits can never exist without a backing project.
- Aggregate totals: SUM(credit_amount) per user (wallet balance), SUM(hectares) per user or per species, COUNT(projects) per community, for the Dashboard's headline metrics.

6.2 Non-Functional Requirements

Quality	Requirement, tied to an observed gap in the current system
Performance	Indexed foreign keys give fast, targeted lookups for wallet/dashboard queries, replacing the current approach of loading the entire upload history into memory on every screen open.
Data integrity	NOT NULL, CHECK and foreign-key constraints replace today's unchecked fields — e.g. nothing currently stops a negative hectares value or an arbitrary species name from being stored.
Security	A password-hash column and real credential verification replace the current mock login, which accepts any non-empty password.
Scalability	A relational table scales to thousands of rows per user without the read-entire-list-then-rewrite-entire-list pattern the app currently uses for every single upload.
Reliability	Database transactions remove the race condition in the current upload flow (read-then-write is not atomic today), where two concurrent uploads could overwrite each other.

7. Database Selection and Justification

Selected technology: SQLite, accessed through the Android Room Persistence Library *— an embedded relational database.*

Why Room/SQLite fits this specific project (not chosen by default, but because the project's own characteristics point to it):

- **Already the intended direction for this app.** The project's build configuration already lists the Room persistence library as a dependency, and the codebase documents it as "prepared for future use." Adopting it is the change with the least unnecessary restructuring, matching the assessment's instruction to prioritise modifications with the most DBMS relevance for the least disruption.
- **The app is offline-first by design.** BlueLedger is explicitly built around local, offline data persistence with no backend server. A cloud document store (Firestore, MongoDB Realm) would introduce a network dependency the product does not currently have; an embedded relational engine keeps that design intact.
- **The data is inherently relational, not document-shaped.** A User has many Projects; a Project references exactly one Species; a Project generates many CreditTransactions. These are clean one-to-many foreign-key relationships with a fixed, shallow shape — not deeply nested or schema-varying documents that would benefit from a document model.
- **Aggregation is a core requirement.** The Dashboard/Wallet screens need SUM and GROUP BY-style aggregates (total hectares, total credits, per-species totals). SQL expresses these directly; a key-value or document store would require pulling every record into the application to aggregate them, which is exactly the current (broken) approach.
- **Transactional guarantees protect the credit ledger.** Minting a credit must never happen without a backing project. A relational database's built-in transaction support gives that guarantee for free; the app's current local-storage mechanism has no transaction concept.
- **Drop-in fit with the existing reactive architecture.** The screens already update reactively from the data layer, so a database that can stream live query results back to the UI (as Room does) requires no change to how screens observe and display data — only the storage layer underneath changes.

7.1 Comparison with alternatives

Option	Type	Verdict for BlueLedger
SQLite / Room (selected)	Embedded relational	Fits offline-first design, relational data shape, existing Flow-based architecture, and an already-declared dependency. Chosen.
Firebase Firestore / MongoDB	Cloud document	Would suit a future multi-device sync feature, but adds a network dependency and cost the current offline-first product does not need; harder to enforce a single source of truth for species rates. Reasonable Phase-2 addition (see Future Enhancements), not a replacement now.
DataStore Preferences (current)	Key-value	Already tried in this repository and shown to be insufficient: no querying, no relations, one overwritable blob per record type, no concurrency safety.
Server-side PostgreSQL/MySQL	Client-server relational	Appropriate once a backend for project verification and a credit marketplace exists (already identified as a known gap). Out of scope until that backend is built.
Graph database (e.g. Neo4j)	Graph	No multi-hop relationship-traversal use case exists in this domain (no social graph, no path-finding); not justified.

7.2 Advantages and limitations

- **Advantages:** zero network dependency, ACID transactions, compile-time-checked SQL via Room annotations, native Flow support, mature Android tooling (Database Inspector), no additional cost.
- **Limitations (acknowledged honestly):** SQLite is a single-writer, file-based engine local to one device — it has no built-in multi-device replication. If BlueLedger later needs data shared across a contributor's devices or a server-side verification workflow, a sync layer or a migration to a client-server database (PostgreSQL) would be required; this is listed under Future Enhancements rather than solved here, to avoid unnecessary restructuring in this phase.

7.3 Integration with the existing stack

The proposed database slots directly underneath the app's existing data layer, which already separates the screens from how data is stored. The behavior the screens currently rely on — logging in, submitting a plot, and reading back totals — stays the same; only the storage mechanism behind it changes, from unstructured local text to a structured, constrained database. As a result, no visible screen or navigation flow needs to change for this migration.

8. Data Model / Database Design

This is the core design section (4 of 10 assessment marks). It presents the ER diagram, per-entity attribute definitions, the relational schema as SQL DDL, key/constraint reasoning, and the normalization argument, all derived from the entities identified above.

8.A Entity-Relationship Diagram

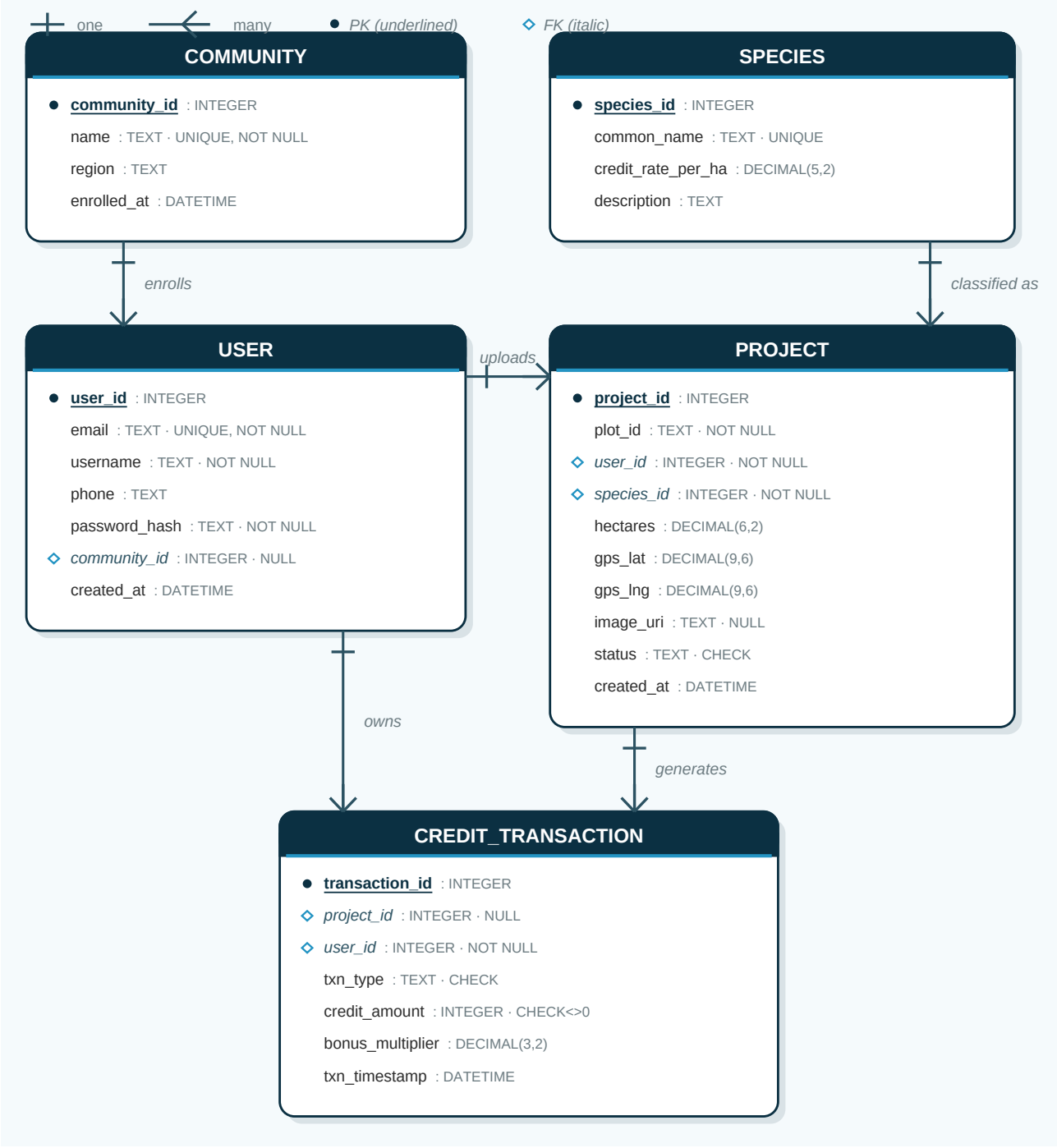

Figure 1. ER diagram for the proposed BlueLedger schema — 5 entities, all one-to-many relationships, surrogate integer primary keys.

8.B Entity Descriptions

COMMUNITY

Represents a village or cooperative enrolled in the program. Newly proposed: the current Dashboard screen already displays a “total communities enrolled” figure with no underlying record to back it.

Attribute	Data Type	Constraint
community_id	INTEGER	PRIMARY KEY, AUTOINCREMENT
name	TEXT	NOT NULL, UNIQUE
region	TEXT	NULLable
enrolled_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

USER

Evolves the current, minimal user record (email, phone, username only) into a real account record with credential storage and optional community membership.

Attribute	Data Type	Constraint
user_id	INTEGER	PRIMARY KEY, AUTOINCREMENT
email	TEXT	NOT NULL, UNIQUE
username	TEXT	NOT NULL
phone	TEXT	NULLable
password_hash	TEXT	NOT NULL
community_id	INTEGER	FOREIGN KEY → COMMUNITY(community_id), NULLable
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

SPECIES

Reference/lookup table. Replaces the hardcoded species-to-rate mapping currently built into the app's credit-calculation logic with queryable, editable data. Seed rows: Rhizophora (14), Avicennia (12), Bruguiera (10) — the rates already used by the app today.

Attribute	Data Type	Constraint
species_id	INTEGER	PRIMARY KEY, AUTOINCREMENT
common_name	TEXT	NOT NULL, UNIQUE
credit_rate_per_ha	DECIMAL(5,2)	NOT NULL, CHECK (credit_rate_per_ha > 0)
description	TEXT	NULLable

PROJECT

Direct evolution of the existing project record. The free-text species field becomes a validated foreign key, and a new status column formalizes a verification workflow that today does not exist (credits are minted instantly and unconditionally).

Attribute	Data Type	Constraint
project_id	INTEGER	PRIMARY KEY, AUTOINCREMENT
plot_id	TEXT	NOT NULL
user_id	INTEGER	FOREIGN KEY → USER(user_id), NOT NULL

Attribute	Data Type	Constraint
species_id	INTEGER	FOREIGN KEY → SPECIES(species_id), NOT NULL
hectares	DECIMAL(6,2)	NOT NULL, CHECK (hectares > 0)
gps_lat	DECIMAL(9,6)	NOT NULL
gps_lng	DECIMAL(9,6)	NOT NULL
image_uri	TEXT	NULLable
status	TEXT	NOT NULL, CHECK IN ('PENDING','VERIFIED','REJECTED'), DEFAULT 'PENDING'
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

CREDIT_TRANSACTION

An append-only ledger. The minted-credit amount is no longer stored solely on the Project record; instead every mint (and, later, every redemption) is its own row. project_id is nullable because a REDEEM transaction draws from a user's pooled balance rather than a single project. Wallet balance = SUM(credit_amount) grouped by user.

Attribute	Data Type	Constraint
transaction_id	INTEGER	PRIMARY KEY, AUTOINCREMENT
project_id	INTEGER	FOREIGN KEY → PROJECT(project_id), NULLable
user_id	INTEGER	FOREIGN KEY → USER(user_id), NOT NULL
txn_type	TEXT	NOT NULL, CHECK IN ('MINT','REDEEM')
credit_amount	INTEGER	NOT NULL, CHECK (credit_amount <> 0)
bonus_multiplier	DECIMAL(3,2)	NOT NULL, DEFAULT 1.00
txn_timestamp	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

8.C Relational Schema (SQL DDL)

The Room persistence library generates SQL equivalent to this from the entity definitions; it is shown directly here for clarity and portability.

```
CREATE TABLE community (
    community_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT NOT NULL UNIQUE,
    region        TEXT,
    enrolled_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user (
    user_id        INTEGER PRIMARY KEY AUTOINCREMENT,
    email          TEXT NOT NULL UNIQUE,
    username       TEXT NOT NULL,
    phone          TEXT,
    password_hash  TEXT NOT NULL,
    community_id   INTEGER,
    created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (community_id) REFERENCES community(community_id)
);

CREATE TABLE species (
    species_id           INTEGER PRIMARY KEY AUTOINCREMENT,
    common_name          TEXT NOT NULL UNIQUE,
    credit_rate_per_ha   DECIMAL(5,2) NOT NULL CHECK (credit_rate_per_ha > 0),
    description          TEXT
);

CREATE TABLE project (
    project_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    plot_id      TEXT NOT NULL,
    user_id      INTEGER NOT NULL,
    species_id   INTEGER NOT NULL,
    hectares     DECIMAL(6,2) NOT NULL CHECK (hectares > 0),
    gps_lat      DECIMAL(9,6) NOT NULL,
    gps_lng      DECIMAL(9,6) NOT NULL,
    image_uri    TEXT,
    status       TEXT NOT NULL DEFAULT 'PENDING'
                 CHECK (status IN ('PENDING','VERIFIED','REJECTED')),
    created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)    REFERENCES user(user_id),
    FOREIGN KEY (species_id) REFERENCES species(species_id)
);

CREATE TABLE credit_transaction (
    transaction_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    project_id        INTEGER,
    user_id           INTEGER NOT NULL,
    txn_type          TEXT NOT NULL CHECK (txn_type IN ('MINT','REDEEM')),
    credit_amount     INTEGER NOT NULL CHECK (credit_amount <> 0),
    bonus_multiplier  DECIMAL(3,2) NOT NULL DEFAULT 1.00,
    txn_timestamp     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (project_id) REFERENCES project(project_id),
    FOREIGN KEY (user_id)    REFERENCES user(user_id),
    CHECK ((txn_type = 'MINT' AND project_id IS NOT NULL) OR txn_type = 'REDEEM')
);

CREATE INDEX idx_project_user     ON project(user_id);
CREATE INDEX idx_project_species  ON project(species_id);
CREATE INDEX idx_txn_user         ON credit_transaction(user_id);
CREATE INDEX idx_txn_project      ON credit_transaction(project_id);
```

Representative queries

```
-- Wallet balance and total hectares for the signed-in user (Wallet / Dashboard screens)
SELECT COALESCE(SUM(ct.credit_amount), 0) AS wallet_balance
FROM   credit_transaction ct
WHERE  ct.user_id = :userId;

SELECT COALESCE(SUM(p.hectares), 0) AS total_hectares
FROM   project p
WHERE  p.user_id = :userId;

-- Per-species totals for an analytics/report view
SELECT s.common_name, COUNT(p.project_id) AS plots, SUM(p.hectares) AS hectares
FROM   project p JOIN species s ON s.species_id = p.species_id
GROUP  BY s.common_name;

-- Community leaderboard by total credits minted
SELECT c.name, SUM(ct.credit_amount) AS total_credits
FROM   credit_transaction ct
JOIN   user u ON u.user_id = ct.user_id
JOIN   community c ON c.community_id = u.community_id
WHERE  ct.txn_type = 'MINT'
GROUP  BY c.name
ORDER  BY total_credits DESC;
```

8.D Keys and Constraints

Constraint type	Where applied	Purpose
PRIMARY KEY	*_id on every table	Uniquely identifies each row; SQLite auto-generates a monotonic surrogate key.
FOREIGN KEY	user.community_id, project.user_id, project.species_id, credit_transaction.project_id/user_id	Guarantees every reference points to a row that actually exists — an integrity check the current unstructured storage cannot make.
UNIQUE	user.email, community.name, species.common_name	Prevents duplicate accounts, duplicate community records, and duplicate species reference rows.
NOT NULL	email, username, password_hash, hectares, gps_lat/lng, txn_type, credit_amount, etc.	Blocks incomplete records — today nothing stops an empty species name from being saved.
CHECK	hectares > 0, credit_rate_per_ha > 0, credit_amount <> 0, status/txn_type enumerations, MINT→project_id required	Encodes domain rules directly in the schema instead of relying on UI-side validation alone.

8.E Normalization

1NF — fixing the current violation

The existing storage is not even in First Normal Form: the entire list of projects is held as one serialized block of text under a single stored value — a repeating group packed into a single atomic field. The proposed schema gives every project its own row with atomic columns, satisfying 1NF.

2NF — trivially satisfied

Every table in the design uses a single-column surrogate primary key (`*_id`), so there are no composite keys and therefore no possibility of a partial-key dependency. 2NF holds by construction.

3NF — the species-rate example

This is the concrete anomaly the redesign removes. If a credit-rate column were added directly onto the project table instead of extracting species into its own table, it would depend on the project's species rather than on the project's

own primary key — a transitive dependency (project → species → credit rate) that violates 3NF. The practical symptom is exactly the situation already present in the app today: the rate is duplicated in application logic rather than being looked up once. Extracting SPECIES as its own table with a foreign key from PROJECT removes the transitive dependency and the update anomaly it would otherwise cause (correcting Rhizophora's rate would no longer require touching every historical project row).

---

9. System Architecture / Workflow Diagram

The diagram below follows the required Users → Application/System → Database → Output flow, mapped onto BlueLedger's actual application layering.

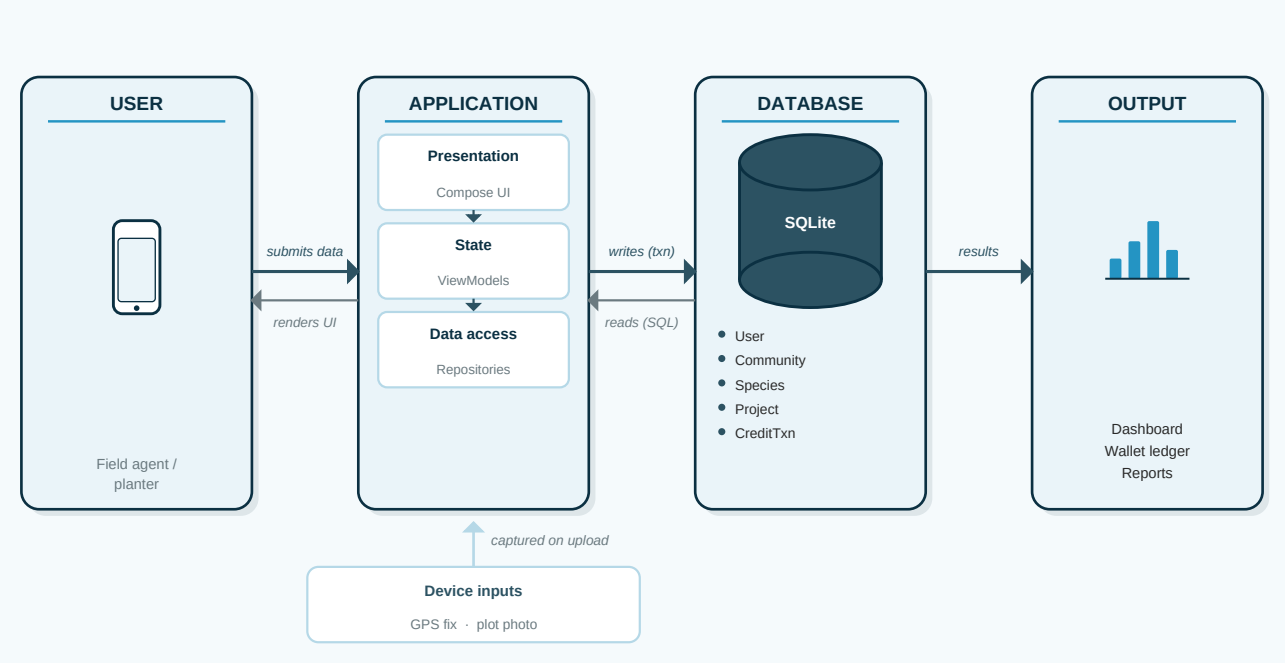

Figure 2. BlueLedger system architecture with the proposed SQLite persistence layer. Solid arrows are write/request calls; the lighter return arrows are the live data updates that refresh the screens.

A. Existing vs Proposed System

Aspect	Existing (today)	Proposed
Storage	Two unstructured text blocks in local device storage	5-table relational schema in an embedded SQLite database
User model	Single overwritable user record, mock authentication	USER table, unique email, password hash, multi-account capable
Species rates	Hardcoded in application logic	SPECIES reference table, editable without a rebuild
Credit history	Baked into each project's single credit-amount field	Append-only CREDIT_TRANSACTION ledger, supports future redemption
Integrity	None — any value accepted	Primary-key, foreign-key, NOT NULL and CHECK constraints enforced by the database engine
Queries	Full-list loading into memory, then in-app filtering	Indexed SQL queries and aggregates (SUM, GROUP BY)
Concurrency	Read-then-write race during upload	Atomic database transactions on insert

B. Proposed Implementation Plan

Phase	Work	Outcome
1. Database setup	Introduce the embedded database and its configuration into the application.	A working, empty database ready to hold data
2. Schema / model implementation	Define the five entities (User, Community, Species, Project, CreditTransaction) with the constraints in §8.D; seed the Species reference rows.	The full schema created and seeded
3. Data-access integration	Implement the queries needed to create, read, update, and delete records, equivalent to today's data-access methods.	All required database operations available
4. Application integration	Connect the existing data layer to the new database instead of the current unstructured local storage, keeping the same behavior visible to the screens.	Screens read/write through the new database transparently
5. Reference-data wiring	Point the species-selection dropdown at the live Species table instead of a fixed list.	Species list becomes data-driven
6. Testing and validation	Test all CRUD operations against a test database; verify constraint violations are rejected; verify wallet totals match manually computed values.	Verified, constraint-correct database behavior

C. Assessment Readiness Checklist

Assessment 6 requirement	Status	Where addressed
1. Project Title	Done	Section 1
2. Introduction and Background	Done	Section 2
3. Problem Statement	Done	Section 3
4. Objectives	Done	Section 4
5. Scope of the Project	Done	Section 5

Assessment 6 requirement	Status	Where addressed
6. Requirement Analysis (functional + non-functional)	Done	Section 6
7. Database Selection and Justification	Done	Section 7
8. Data Model / Database Design (ER diagram, entities, schema, keys, normalization)	Done	Section 8
9. System Architecture / Workflow Diagram	Done	Section 9
Distinguishes implemented vs proposed features	Done	Sections 2, 3, 5, Appendix A
Implementation not yet performed on the live codebase	Confirmed — no source files were modified	This document is a proposal only

D. Future Enhancements

- **Verifier role and review screen:** build the screen that lets a second role transition a project's status from PENDING to VERIFIED/REJECTED before a credit-mint transaction is allowed to fire.
- **Cloud sync:** once multi-device access is needed, add a thin sync layer (or move to a server-backed relational database) rather than switching the local model — the existing foreign-key-based schema maps cleanly onto a server-side PostgreSQL schema if that becomes necessary.
- **Credit redemption workflow:** implement the already-planned “Sell Credits” action on the Wallet screen using REDEEM rows in the credit-transaction ledger.
- **Historical rate versioning:** if species credit rates change over time, add an effective-date range to the Species table (or a separate rate-history table) so past mints remain reproducible.

These are explicitly out of scope for the current submission and are listed only to show the schema was designed with room to grow, per the assessment's request for realistic future extensions.